

KVGate performance analysis: LMCache MP mode (CPU L1 + Redis L2)

Rishika Vaish

June 2026

This report measures KVGate, an open-source multimodal LLM gateway, running on LMCache in multiprocess (MP) mode with CPU as the L1 cache and Redis as the L2 cache.

KVGate has two parts, and we measured both:

1. **The cache layer (LMCache).** Each model replica keeps recently-used KV (the model’s computed “memory” of a prompt) so it does not recompute it. There are two tiers: L1 in CPU RAM (fast, local to one replica) and L2 in Redis (shared across replicas, survives restarts).
2. **The routing layer (KVGate).** KVGate sits in front of the replicas and chooses which replica each request goes to. Its main strategy, `prefix_kv_aware`, sends a request to the replica that already holds the longest matching prefix, so the cache actually gets used.

The short version: KVGate’s routing picks the replica that already has the answer warm, and LMCache’s L1 plus L2 makes that warm hit cheap and durable. Below we show what each one contributes, with measured numbers, including the cases where they do not help.

The four numbers that matter

Metric	Plain meaning	Result
Routing affinity	How often a request lands on a replica that already holds its prefix	39% to 87% at 8 replicas (sim); 99% on the live GPU run
TTFT	How long the user waits for the first token	1066 ms down to 383 ms with KVGate routing (about 2.8x faster)
Throughput	Requests served per second	+19% from routing; +46% from Redis L2 once the cache overflows the GPU
Cache hit rate	Share of KV served from cache instead of recomputed	74% to 97% after a restart; about 99% under load

Setup: vLLM 0.23.0, LMCache 0.5.0, torch 2.11.0+cu130, CUDA 13.0, Python 3.12, model `llava-hf/llava-onevision-qwen2-7b-ov-hf`, on NVIDIA A40 GPUs. Every cache-layer con-

fig was run cold, then warm three times, so the tables below are averages with a measured spread, not a single lucky run. Reproduction and raw data: [benchmarks/lmcache_mp/](#), [benchmarks/kvgate_routing/](#), [results/](#).

What changed: in-process to MP mode

KVGate is a router. It proxies requests to vLLM replicas and does not run the model or the KV cache itself (its package depends on fastapi, httpx, and pydantic (see pyproject.toml), not on vllm, lmcache, or torch). LMCache runs inside the vLLM replicas, so the move to MP mode, the Redis L2 tier, and the library versions are all configured at the vLLM launch layer, not in KVGate’s router code.

Dimension	Before (in-process)	After (MP mode)
LMCache connector	LMCacheConnectorV1 (inside vLLM)	LMCacheMPConnector (separate server)
Cache tiers	L1 = CPU only	L1 = CPU plus L2 = Redis
Config file	benchmarks/lmcache_cpu.yaml	benchmarks/lmcache_mp.yaml
vLLM	0.11.0	0.23.0
LMCache	0.3.7	0.5.0
torch / CUDA	cu128 / 12.8	2.11.0+cu130 / 13.0
Gateway	single replica	two replicas (config/gpu.runpod.yaml)

The before and after architecture:

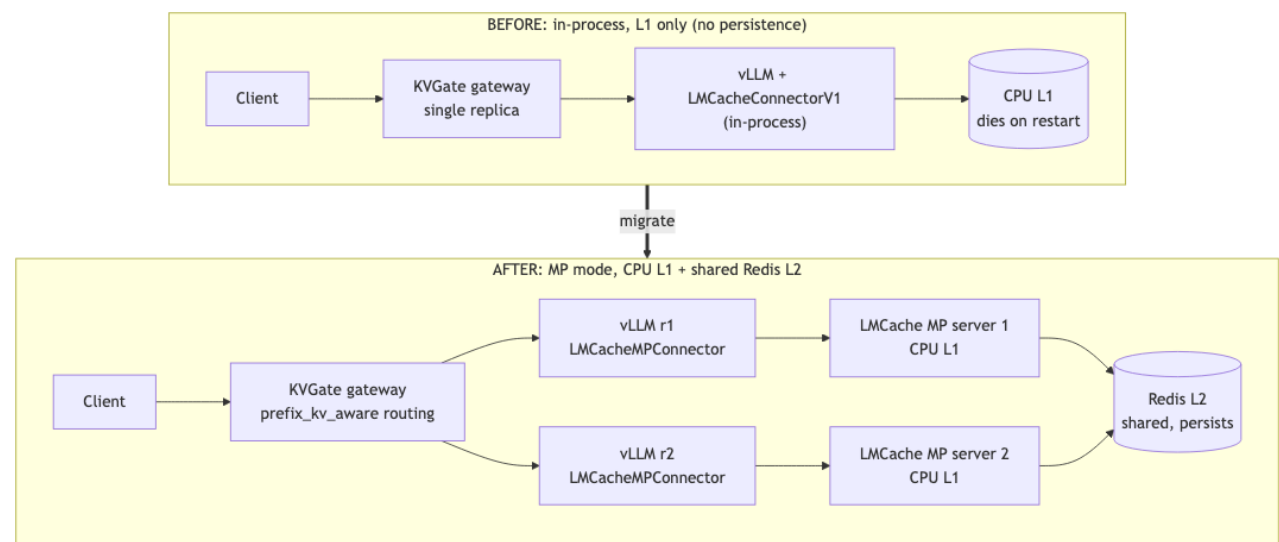


Figure 1: Architecture: before and after

The exact launch difference. Before (in-process, L1 CPU only):

```
LMCACHE_CONFIG_FILE=benchmarks/lmcache_cpu.yaml \  
vllm serve $MODEL \  
  --kv-transfer-config \  
  '{"kv_connector":"LMCacheConnectorV1","kv_role":"kv_both"}'
```

After (MP mode, L1 CPU plus Redis L2):

```
lmcache server --l1-size-gb 2 \  
  --l2-adapter '{"type":"resp","host":"127.0.0.1","port":6379}' \  
vllm serve $MODEL \  
  --kv-transfer-config \  
  '{"kv_connector":"LMCacheMPConnector","kv_role":"kv_both"}'
```

The MP server is a separate process holding the two-tier cache, and vLLM attaches as a client. This is what enables the Redis L2 tier and cross-restart persistence (Part 1 Finding 1, and Part 3), neither of which in-process mode can provide.

Where each change lives in the repo. The “before” column is the repo’s pre-existing state (benchmarks/lmcache_cpu.yaml and docs/BENCHMARK_REPORT.md); the “after” column is this PR.

- Connector and L2 tier: the `kv-transfer-config` and `l2-adapter` flags in benchmarks/lmcache_mp/run.sh and benchmarks/kvgate_routing/run_gpu_2replica.sh.
- Cache config: benchmarks/lmcache_cpu.yaml (in-process L1) becomes benchmarks/lmcache_mp.yaml (MP, L1 + L2).
- Library versions: benchmarks/requirements-mp.txt.
- Two-replica gateway: config/gpu.runpod.yaml.

Part 1: the cache layer (LMCache L1 + L2)

We ran one vLLM replica plus one LMCache MP server (CPU L1 + Redis L2). We capped the GPU at `--gpu-memory-utilization 0.5` on purpose. If we left the GPU wide open, vLLM’s own on-GPU cache would hold the whole working set and any off-GPU cache would sit idle and look pointless. Capping it forces the working set to overflow the GPU, which is the real situation for big models and long multimodal prompts. The verified overflow here was about 9.8 GB of KV against a 2 GB L1.

Finding 1: only Redis L2 survives a restart

The GPU cache and the CPU L1 both live inside the model process. Restart or redeploy that process and they are gone. Redis is a separate process, so it is not. We tested this directly: fill the cache, then kill both vLLM and the LMCache server (wiping the GPU cache and CPU L1) while leaving Redis running, restart, and replay the same traffic.

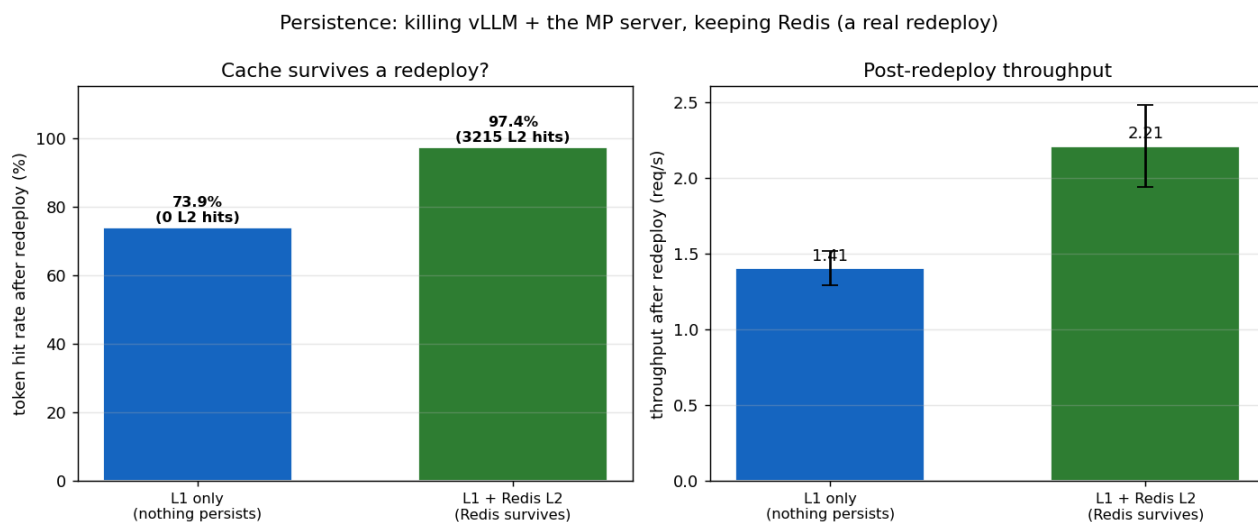


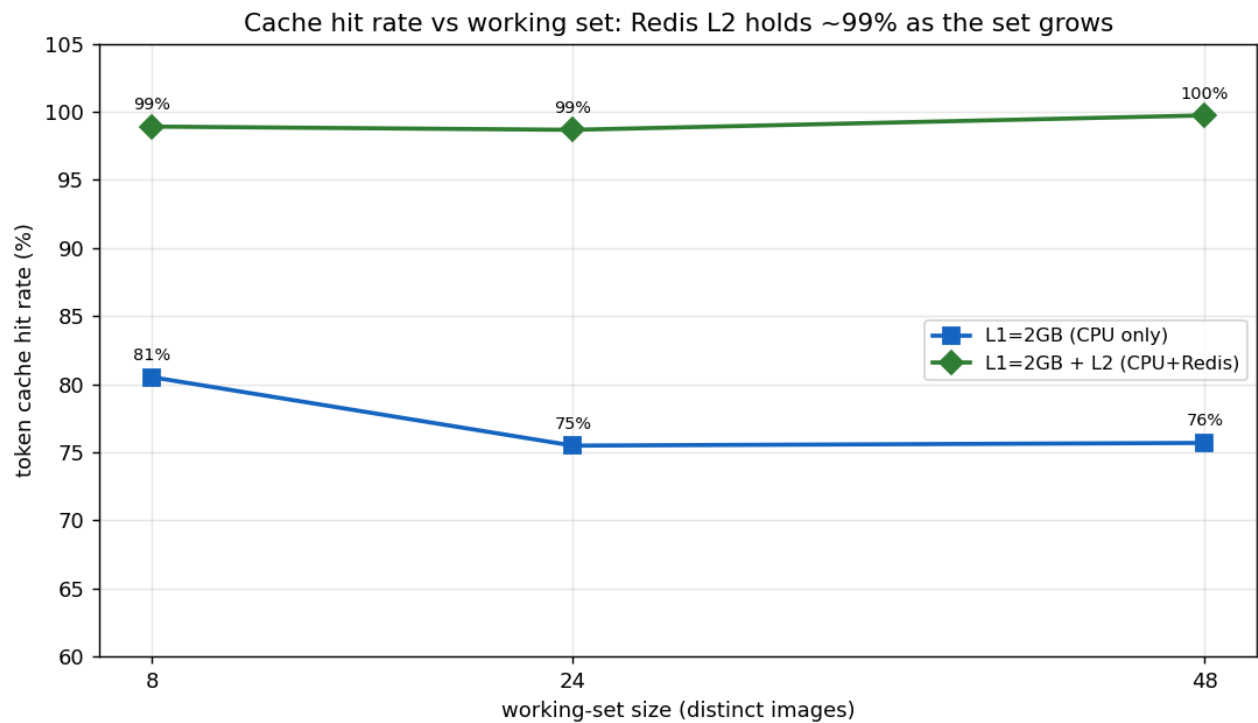
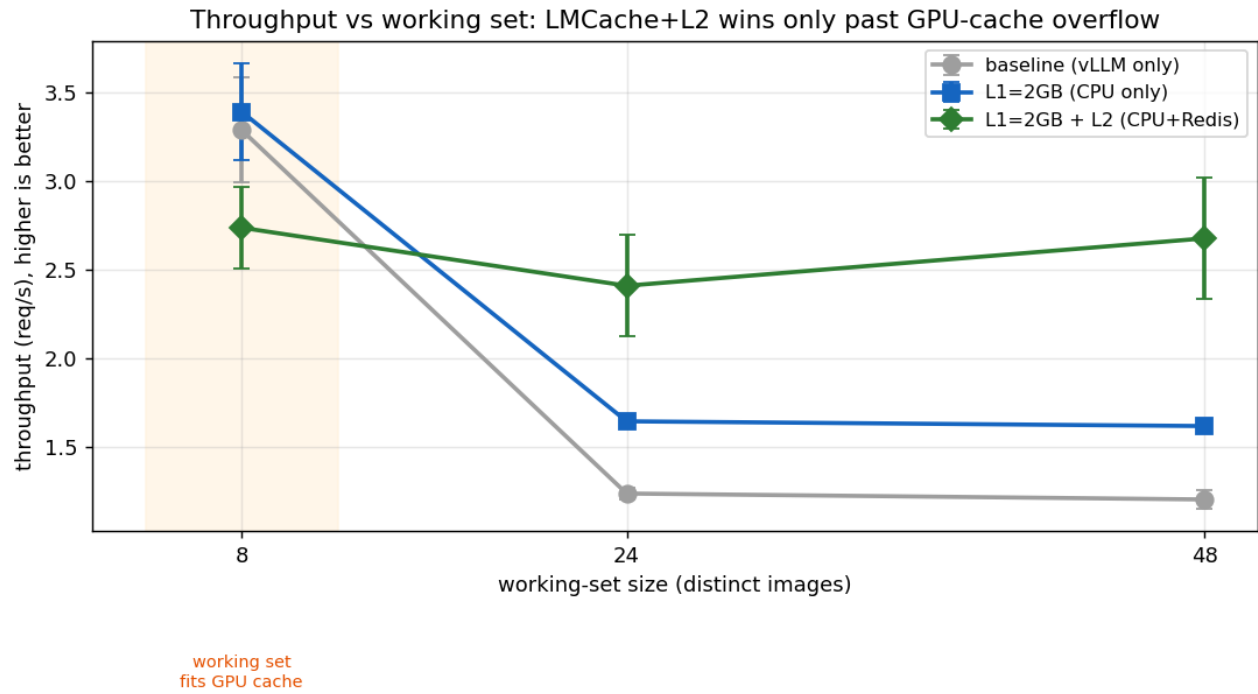
Figure 2: Persistence after redeploy

After a real restart	Cache hit rate	KV chunks served from	
		Redis	Throughput
L1 only (nothing survives)	73.9%	0	1.41 req/s
L1 + Redis L2	97.4%	3,215	2.21 req/s

The 3,215 versus 0 is the whole point. A freshly restarted server, pointed at the same Redis, found and served the old KV. It works because LMCACHE names each cache entry by a hash of its content, so the same request recomputes the same key and finds it in Redis with nothing to rebuild. The result is about 1.6x the throughput and roughly half the tail latency right after a restart.

Why this matters: a single throughput number from a server that never restarts hides this completely. In production, replicas restart and scale up and down constantly, and only Redis keeps the cache warm across those events.

Finding 2: Redis L2 pays off once the working set overflows the GPU



Growing the working set from 8 to 48 distinct images:

Working set	No cache	L1 only	L1 + L2	L1-only hit rate	L1+L2 hit rate
8 images (fits GPU)	3.29	3.39	2.74	81%	99%
24 images (overflows)	1.24	1.65	2.41	75%	99%
48 images (overflows)	1.21	1.62	2.68	76%	99%

Once the set overflows the GPU, the no-cache baseline falls to about 1.2 req/s because vLLM keeps recomputing KV it cannot hold on the GPU. CPU L1 alone recovers some of it but flattens at about 75% hit rate, because its 2 GB fills up and starts evicting. Adding Redis L2 lifts the hit rate to about 99% and throughput to about 2.6 req/s, and holds it flat as the set keeps growing, because Redis absorbs the overflow that the small L1 cannot. Throughput follows hit rate almost exactly, which is why the two charts have the same shape.

Finding 3: when the set fits the GPU, the cache is just overhead

At 8 images the working set fits in the GPU's own cache, so the GPU already serves every prefix. There, LMCache only adds lookup and transfer work for KV that nobody needed off the GPU:

- No cache and L1-only run at 3.29 and 3.39 req/s, but L1+L2 drops to 2.74 req/s. The cache is the slowest option here.
- Its 99% hit rate at 8 images is real but wasted, because it re-serves KV the GPU already had.
- The error bars do not overlap, so this slowdown is real, not noise.

The takeaway is a rule, not a default: turn LMCache MP and Redis L2 on when the KV working set is larger than GPU capacity (big models, long multimodal prompts, many tenants), and leave it off when the set fits on the GPU. The deciding factor is the working-set-to-GPU ratio.

Part 2: the routing layer (KVGate)

A cache only helps if requests reach the replica that holds their warm KV. That is KVGate's job. We measured it two ways.

The routing decision on its own (no GPU). We replayed a realistic multimodal trace (200 sessions, 3 turns, 70% image reuse) through KVGate's actual routing code (same prefix hashing as production; only the tokenizer is approximate) and measured locality hit rate: the share of requests sent to a replica that already holds a matching prefix.

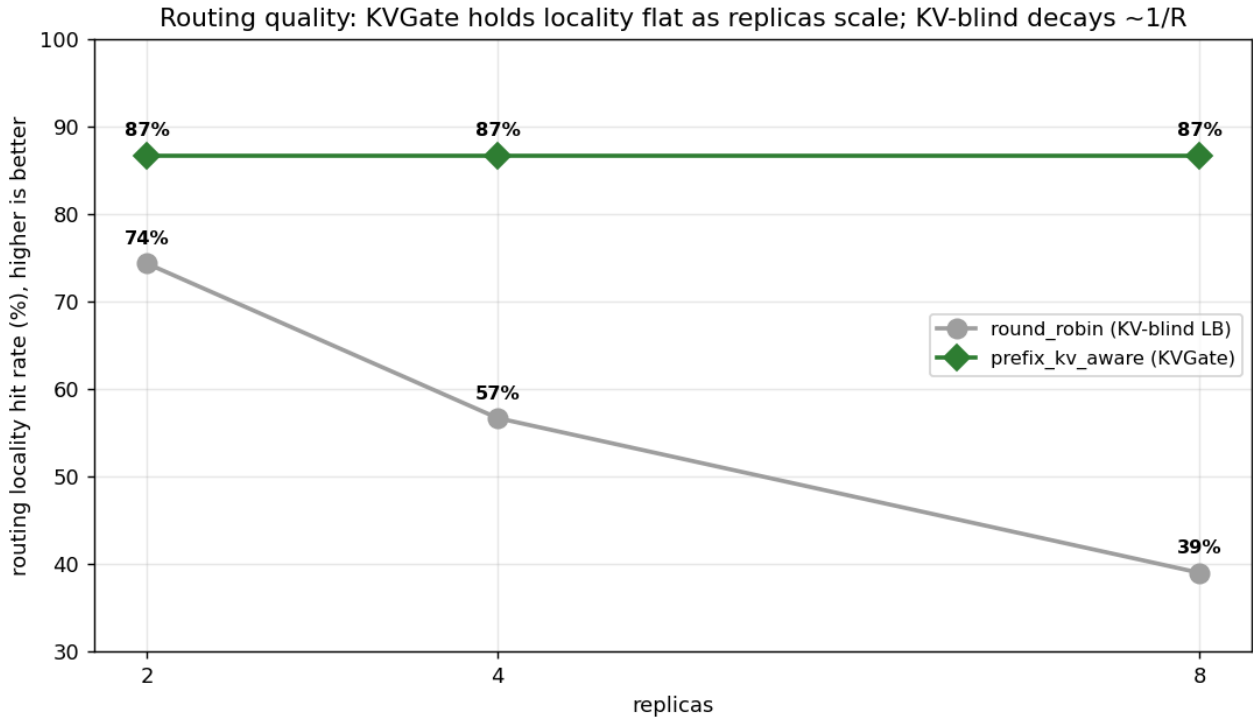


Figure 3: Routing affinity vs replicas

Replicas	round_robin (KV-blind)	prefix_kv_aware (KVGate)
2	74%	87%
4	57%	87%
8	39%	87%

Round-robin loses locality as replicas grow (roughly $1/R$ over the replica count); KVGate holds flat at about 87%. The more you scale out, the more it matters.

Does that become real speed? (live 2-replica GPU run). Two live llava-7B replicas on one A40, each with its own CPU L1 and a shared Redis L2, behind the gateway. Same model, same 180-request trace, a fresh flushed fleet for each run, so routing strategy was the only variable.

Live 2-replica GPU run: same model, same trace, only the routing strategy differs

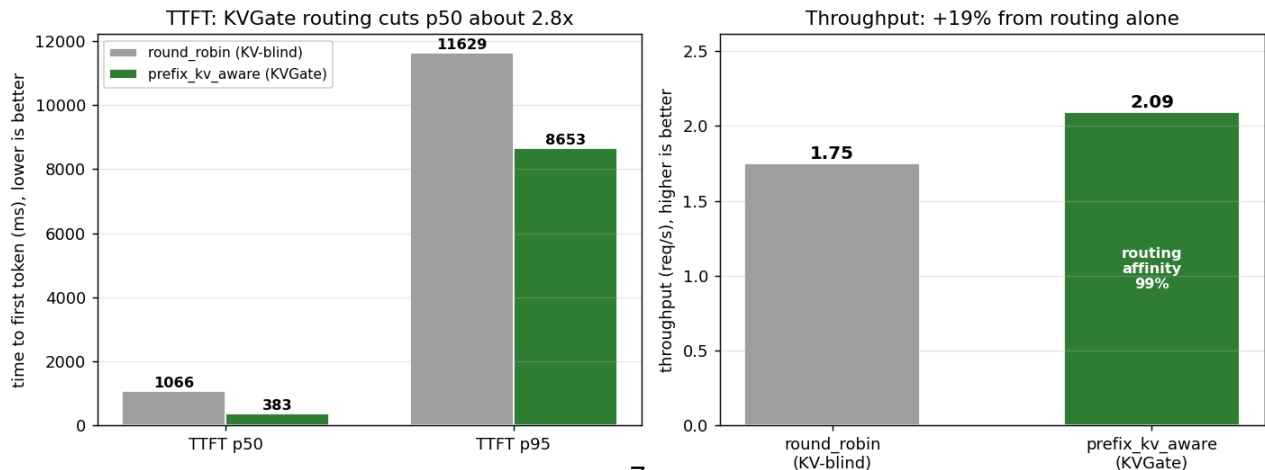


Figure 4: Routing end-to-end

Is MP mode faster per request? No, and it is not meant to be. The runs are not a fair race (different vLLM version, GPU cap, and session shape), and MP mode adds an inter-process hop by design. Its value is what in-process cannot do: **persistence across restarts** (97.4% hit after a restart, Finding 1) and a **shared L2 across replicas** (seen in Part 2). For a multi-replica gateway like KVGate, that is worth a small per-request cost.

What we would improve in KVGate next

1. **Multimodal-aware eviction.** L1 evicts by plain LRU, but one image's vision KV (about 300 MB) is far costlier to recompute than a text turn. Evicting by recompute cost would raise the real hit rate at the same L1 size.
 2. **Warm the cache from Redis on deploy.** Redis survives a restart, but the first request still pays a cold fetch. KVGate could prefetch a replica's hot prefixes at startup.
 3. **Auto-enable L2 by the working-set/GPU ratio.** Finding 3 shows L2 helps only past overflow. KVGate already tracks affinity and could flip L2 on or off itself instead of relying on an operator.
-

Honesty and limits

- The routing sim uses approximate (whitespace) tokenization, faithful to the decision but not byte-identical to the engine's block boundaries.
 - Part 1 is single-replica; cross-replica L2 sharing is shown in Part 2.
 - The GPU was capped on purpose to force overflow; on a bigger GPU or smaller model, Finding 3 takes over and LMCACHE should be off.
 - Redis L2 raises median per-fetch latency; the wins are tail latency, throughput, and persistence.
 - The image corpus is synthetic, but the structure (overflow crossover, persistence, routing locality) comes from the cache hierarchy and routing algorithm, not the specific images.
-

How to reproduce

All scripts live in the repo. Stack: vLLM 0.23.0, LMCACHE 0.5.0 (pin this exact version; an unpinned install resolves to 0.3.7, which predates MP mode and crashes), on a CUDA 13 host. Pins are in benchmarks/requirements-mp.txt.

Routing simulation (no GPU, runs anywhere):

```
python3 benchmarks/kvgate_routing/route_sim.py
```

Cache-layer sweep (one GPU), the Part 1 findings:

```
redis-server --daemonize yes  
tmux new -d -s sweep "bash benchmarks/lmcache_mp/run.sh"
```

Live 2-replica routing run (one GPU), Part 2b:

```
tmux new -d -s gpu2 "bash benchmarks/kvgate_routing/run_gpu_2replica.sh"
```

Operational notes from the GPU runs: pin lmcache==0.5.0; on RunPod, nginx occupies ports 8001 and 8081 so the MP HTTP frontends use 18091 and 18092; vLLM spawns a VLLM::EngineCore subprocess that must be killed explicitly on teardown or the next run runs out of memory; the gateway health endpoint is /healthz.